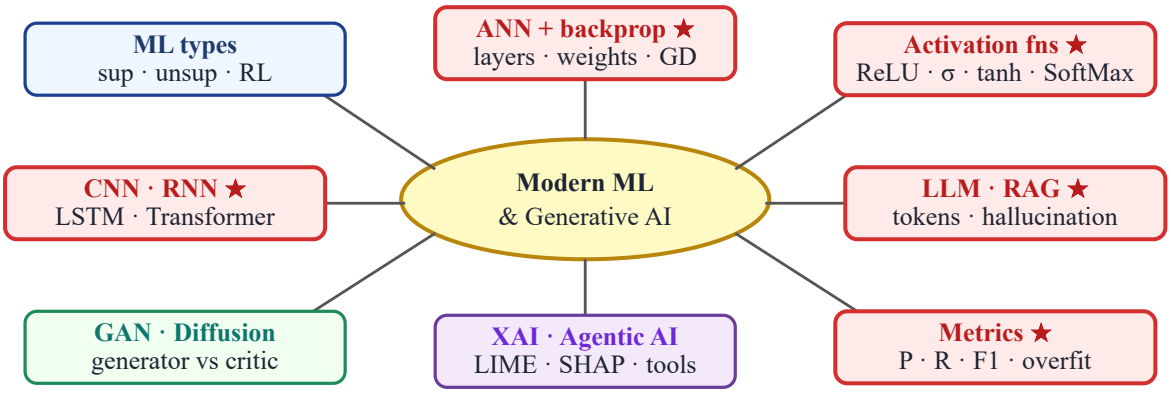
Modern ML & Generative AI

ML types · neural nets · deep learning · LLMs · GenAI · XAI · agents · metrics: exam-ready in 10 sheets



Legend: **yellow**=key term · ●=must-memorise · =memory trick · =exam trap

Syllabus coverage: Machine learning, neural networks, deep learning, generative AI and LLMs, explainable AI, model evaluation. Expert systems, search, Bayes/NLP and regression have their own lessons in this chapter.

Top 3 traps: ① sigmoid is (0,1), tanh is (−1,1)
② RAG reduces, never removes, hallucination
③ high accuracy can hide recall = 0

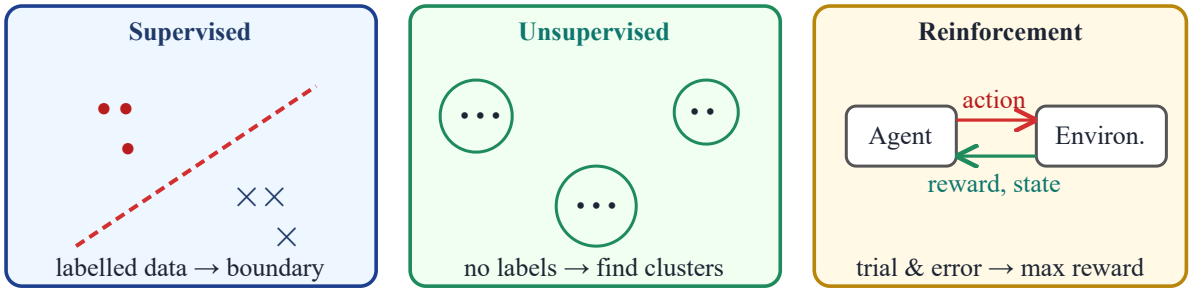
High-yield list
① time-series → RNN / LSTM
② images → CNN
③ backprop: MLP, CNN, RNN = all
④ softmax = multiclass output
⑤ GAN = generator vs discriminator
⑥ LIME local · SHAP Shapley
⑦ F1 = harmonic mean of P, R

Contents

1. ML types & transfer learning … p.2
2. ANN, backprop & gradient descent … p.3
3. Activation functions … p.4
4. Deep learning: CNN · RNN · Transformer … p.5
5. LLMs, RAG & prompting … p.6
6. Generative AI & XAI … p.7
7. Agentic AI · over/underfitting … p.8
8. Evaluation metrics … p.9
9. Night-before + practice … p.10

1 Machine Learning & Its Three Types ★ always asked

Definition: **Machine Learning (ML)** is the branch of AI in which a system **learns patterns from data** and improves with experience, instead of being explicitly programmed with rules. AI ⊃ ML ⊃ Deep Learning.



Type	Learns from	Algorithms	Examples
Supervised	labelled (x, y)	regression, SVM, tree, KNN	spam filter, fraud
Unsupervised	unlabelled x	K-means, PCA, Apriori	customer segments
Reinforcement	reward signal	Q-learning	AlphaGo, robots
Self-supervised	labels made from data	next-token prediction	LLM pre-training

Memory trick: **Teacher · Explorer · Player**: supervised has a *teacher* (labels), unsupervised is an *explorer* (groups it finds itself), reinforcement is a game *player* (score = reward).

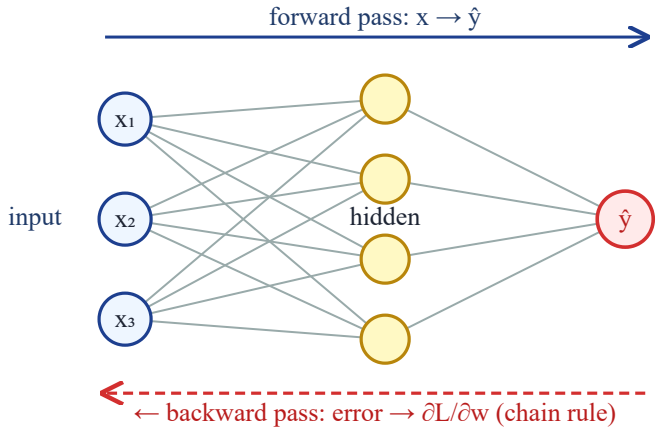
Transfer learning: Take a **pre-trained model** (trained on a huge dataset) and **fine-tune** it on a small task-specific dataset. Saves data, time and GPU cost. e.g. BERT → fine-tuned for **Bangla sentiment analysis**; an ImageNet CNN → X-ray classifier.

Exam trap: Classification & regression = **supervised**; clustering = **unsupervised**. **K-means ≠ KNN**: K-means clusters unlabelled data; KNN classifies using labelled neighbours.

Worked example: Classify: (i) house price from size → supervised (regression); (ii) grouping mobile-banking users by behaviour → unsupervised; (iii) traffic-signal timing that learns from queue lengths → reinforcement; (iv) flagging fraud from past labelled cases → supervised (classification).

2 ANN, Backpropagation & Gradient Descent ★ top cluster

ANN: A computational model inspired by biological neurons: **input layer → hidden layer(s) → output layer**. Every connection carries a **weight**; each neuron adds a **bias** and applies an **activation function**. Training = finding weights that minimise the loss.



One neuron (perceptron)

$z = \Sigma\, w_i x_i + b$

output = f(z)

update: $w \leftarrow w - \eta \cdot \partial L/\partial w$

η = learning rate

L = loss (MSE / cross-entropy)

1. **Forward pass**: inputs flow layer by layer to give the prediction ŷ.
2. **Loss**: compare ŷ with true y (MSE / cross-entropy).
3. **Backpropagation**: the **chain rule** sends the error backwards and gives $\partial L/\partial w$ for every weight.
4. **Gradient descent**: step each weight against its gradient; repeat for many **epochs**.

Worked example: $x_1=1$, $x_2=0$, $w_1=0.5$, $w_2=0.3$, $b=0.1$, sigmoid. $z = 0.5\times1 + 0.3\times0 + 0.1 =$ **0.6**; $\sigma(0.6) = 1/(1+e^{-0.6}) = 1/(1+0.549) \approx$ **0.646**.

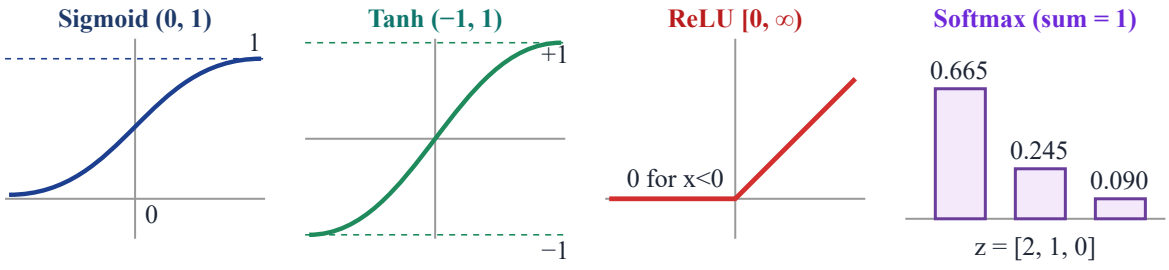
Point	ANN	BNN (brain)
Unit	artificial neuron (maths)	biological neuron (~86 billion)
Signal	numbers, digital	electrochemical spikes
Learning	backpropagation updates weights	synaptic plasticity (Hebbian)
Speed / power	fast per op, high energy (GPU)	slow neurons, ~20 W, massively parallel

Memory trick: **F-L-B-U**: **F**orward · **L**oss · **B**ackward · **U**pdate. Backprop *finds* the gradient; GD *uses* it. η too large → diverges; too small → slow.

Exam trap: "Which model uses backpropagation: MLP / CNN / RNN?" → **All of the above**.

3 Activation Functions ★ written favourite

Why needed: An activation function adds **non-linearity**. Without it every layer is linear and the whole deep net collapses into one linear map: $W_2(W_1x) = (W_2W_1)x$, so depth gives nothing. It also squashes outputs into a useful range (probability, ±1).



Function	Formula	Range	Best use	Weak point
Sigmoid σ	$1 / (1 + e^{-x})$	**(0, 1)**	binary output (probability)	vanishing gradient ($\sigma' \le 0.25$)
Tanh	$(e^{x} - e^{-x})/(e^{x} + e^{-x})$	**(−1, 1)**	hidden layers, RNN; zero-centred	still saturates
ReLU	max(0, x)	[0, ∞)	default for hidden layers; fast	"dying ReLU" (stuck at 0)
Leaky ReLU	max(0.01x, x)	(−∞, ∞)	fixes dying ReLU	extra hyper-parameter
Softmax	$e^{z_i} / \Sigma\, e^{z_j}$	(0, 1), Σ = 1	multiclass output layer	output layer only
Linear	f(x) = x	(−∞, ∞)	regression output	no non-linearity

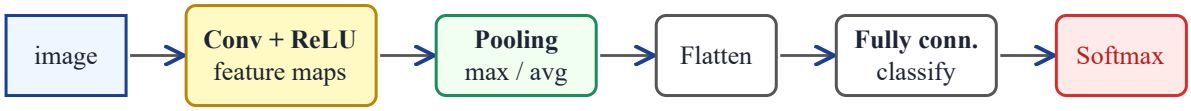
Worked example: Softmax of z = [2, 1, 0]: $e^2 = 7.389$, $e^1 = 2.718$, $e^0 = 1$, sum = 11.107 → **[0.665, 0.245, 0.090]**, total = 1. ReLU(−3) = 0, ReLU(2.5) = 2.5; σ(0) = 0.5; tanh(0) = 0.

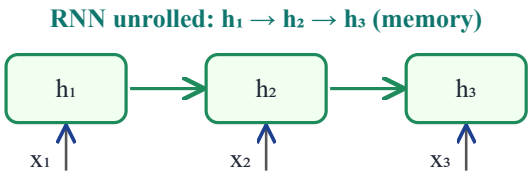
Memory trick: **S**igmoid = **S**ingle yes/no; **S**oftmax = **S**everal classes; **T**anh = **T**wo-sided (±1); **R**eLU = **R**amp.

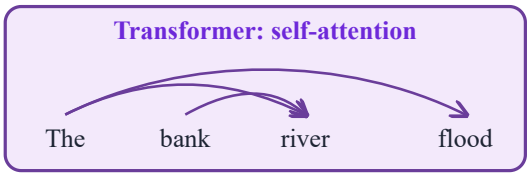
Exam trap: Sigmoid range is **(0, 1)**, not (−1, 1), which is tanh. Deep nets prefer **ReLU** in hidden layers because its gradient is 1 for x > 0, so it largely avoids the **vanishing gradient** problem.

4 Deep Learning: CNN · RNN · Transformer ★ MCQ magnet

Deep learning: ML with **many hidden layers** that learn features automatically from raw data (pixels, words, audio). Needs large datasets and GPUs. Excels on unstructured data.







Model	Data	Key idea	Uses
CNN	images, grids	shared filters slide over input; pooling	face/object recognition, X-ray
RNN	sequences, time-series	hidden state carries memory of past steps	forecasting, speech
LSTM	long sequences	3 gates: forget, input, output; fixes vanishing gradient	stock series, translation
GRU	sequences	2 gates: update, reset; lighter LSTM	same as LSTM
Transformer	text, also images	self-attention over all positions at once; parallel; positional encoding	BERT, GPT, LLMs

Worked example: 32×32 image, 5×5 filter, stride 1, no padding: output = (N − F + 2P)/S + 1 = (32 − 5 + 0)/1 + 1 = **28×28**; then 2×2 max-pool → **14×14**.

Memory trick: **C**NN = **C**amera; **R**NN = **R**emembers order; **T**ransformer = looks at the whole **T**ext at once. Attention(Q,K,V) = softmax($QK^T/\sqrt{d_k}$)·V.

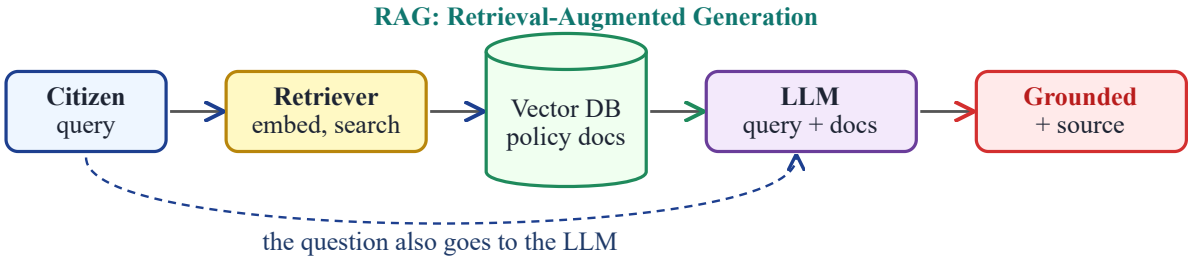
Exam trap: "Best for time-series: CNN / RNN / DNN / ANN?" → **RNN** (LSTM if offered). Transformers replaced RNNs in NLP because they process tokens **in parallel**; BERT = encoder, GPT = decoder.

5 LLMs, RAG, Hallucination & Prompting ★ rising fast

LLM: A **Large Language Model** is a Transformer-based neural network trained on massive text to understand and generate language. It works on **tokens** (word pieces) and repeatedly predicts the **next token**. Examples: GPT (OpenAI), Claude (Anthropic), Gemini (Google), LLaMA (Meta). Uses: chatbots, summarising, translation, code.

1. **Pre-training**: self-supervised next-token prediction on web-scale text.
2. **Fine-tuning**: instruction / task data.
3. **Alignment**: human feedback (RLHF) for helpful, safe answers.

Key terms: **token** = unit of text; **context window** = max tokens seen at once; **temperature** = randomness (low = focused); **embedding** = text as a vector.



Hallucination: The model produces **fluent, confident but false** content (fake citations, wrong dates). Cause: it predicts plausible text, with no built-in fact check. Mitigation: **RAG**, fine-tuning on verified data, citing sources, confidence flags, human review for high-stakes answers.

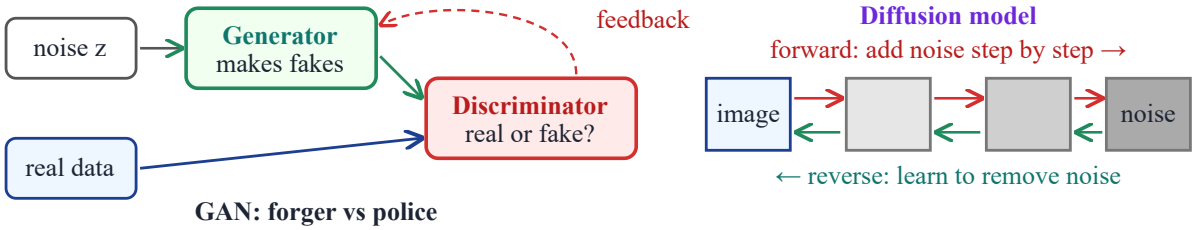
Prompting technique	What you give	Example
Zero-shot	instruction only	"Classify this complaint as tax / land / other."
Few-shot	a few solved examples first	3 labelled complaints, then the new one
Chain-of-thought	ask for step-by-step reasoning	"Work out the VAT step by step."
Role prompting	assign a persona	"You are an NBR help-desk officer…"

Memory trick: Plain LLM = **closed-book exam** (answers from memory, may bluff). RAG = **open-book exam** (looks up the page first, then writes).

Exam trap: RAG does **not** retrain the model (fine-tuning changes weights; RAG only adds context), and it **reduces, not eliminates** hallucination. Hallucination is a by-product of pattern-based generation, not a coding bug.

6 Generative AI: GANs & Diffusion; XAI ★ bank written

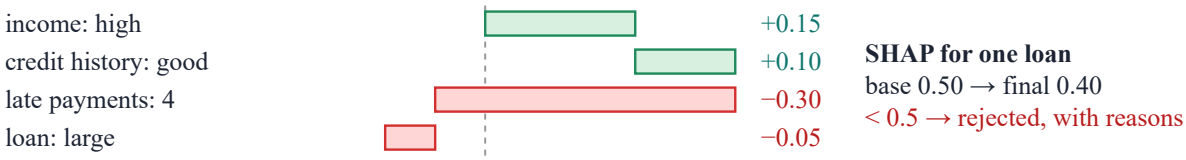
Generative AI: AI that **creates new content**: text (LLMs), images (Stable Diffusion, DALL-E, Midjourney), audio, video, code, synthetic data. A **discriminative** model learns P(y | x) to label data; a **generative** model learns P(x) to make new samples.



GAN: forger vs police

Model	How it generates	Known for
GAN	Generator vs Discriminator, trained adversarially	faces, deepfakes
Diffusion	start from noise, denoise step by step	Stable Diffusion, DALL-E
VAE	encode to latent space, decode samples	smooth variations
LLM	Transformer, next-token prediction	text, code, chat

XAI: **Explainable AI** shows **why** a model gave an output: needed for loans, diagnosis, public services, audit. **LIME** (Local Interpretable Model-agnostic Explanations) perturbs one input and fits a simple local model. **SHAP** (SHapley Additive exPlanations) uses game-theory Shapley values to give each feature a signed contribution. **Saliency maps / Grad-CAM** highlight image regions.

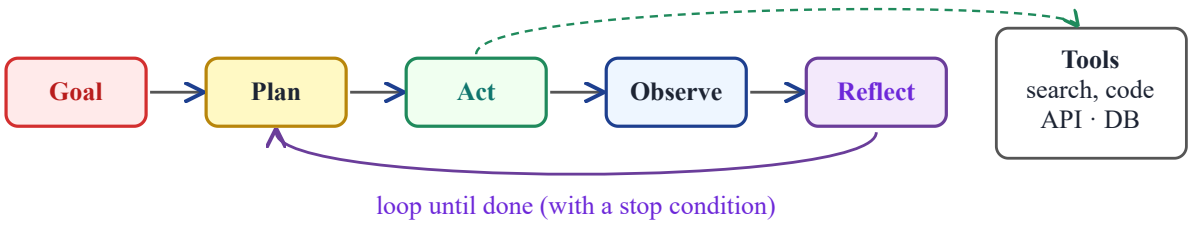


Memory trick: **GAN = forger vs police**. **L**IME = **L**ocal; **SH**AP = **SH**apley. Check: 0.50 + 0.15 + 0.10 − 0.30 − 0.05 = 0.40.

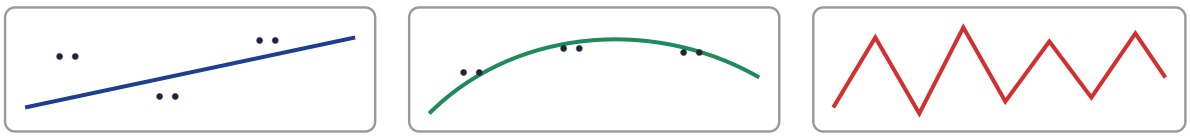
Exam trap: The **Discriminator** judges real vs fake; the Generator never sees real data, it learns only from the Discriminator's feedback. Generative AI is **not only chatbots**.

7 Agentic AI vs Chatbot; Over- & Underfitting ★ BUET favourite

Agentic AI: An AI system that takes a **goal**, plans the steps, **uses tools** (web search, code execution, APIs, databases), observes results, keeps memory and self-corrects until the goal is met, with minimal human help. Core: an LLM as the reasoning engine + tools + memory + an orchestration loop.



Dimension	Traditional chatbot	Agentic AI
Logic	rules / scripts, FAQ matching	LLM reasoning + planning
Turns	single question → answer	**multi-step**, goal-directed
Memory	none across sessions	short + long-term memory
Tools	none	search, code, APIs, files
Self-correction	no	yes: reflect, retry
Risk	wrong canned answer	hallucination chains, tool misuse, endless loops



Underfit: high bias | **Good fit** | **Overfit: high variance**

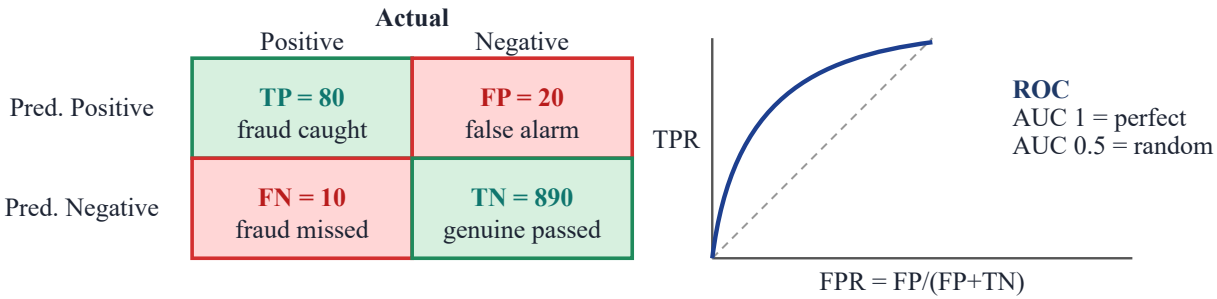
	Train error	Test error	Remedies
Overfitting	very low	high	more data, L1/L2 regularisation, dropout, early stopping, cross-validation, augmentation, simpler model
Underfitting	high	high	bigger model, better features, train longer, less regularisation

Memory trick: Overfit = student who **memorised last year's guide** and fails new questions; underfit = student who **did not study** and fails both.

Exam trap: Overfitting = **high variance, low bias**; underfitting = **high bias, low variance**. Dropout fights overfitting, never underfitting.

8 Confusion Matrix, Precision, Recall, F1 ★ numerical

Confusion matrix: A table of predicted vs actual classes. **TP** = positive predicted positive; **FP** = negative wrongly flagged (false alarm, Type I); **FN** = positive missed (Type II); **TN** = negative correctly left alone.



Metric	Formula	Question it answers
Accuracy	(TP + TN) / total	how many predictions were right?
Precision	TP / (TP + FP)	of those flagged positive, how many were truly positive?
Recall (sensitivity, TPR)	TP / (TP + FN)	of the real positives, how many did we catch?
Specificity (TNR)	TN / (TN + FP)	of the real negatives, how many were left alone?
F1-score	2PR / (P + R)	harmonic mean: balances P and R

Worked example: Fraud model, 1000 transactions: TP = 80, FP = 20, FN = 10, TN = 890.

- Accuracy = (80 + 890)/1000 = **0.97**
- Precision = 80/(80 + 20) = **0.80**; Recall = 80/(80 + 10) = 80/90 = **0.889**
- F1 = 2 × 0.80 × 0.889 / (0.80 + 0.889) = 1.422 / 1.689 ≈ **0.842**; Specificity = 890/910 ≈ 0.978

Memory trick: **P**recision looks at the **P**redicted-positive row; **R**ecall looks at the **R**eal-positive column. Precision matters when false alarms cost (spam filter); recall matters when misses cost (cancer, fraud).

Exam trap: **Accuracy paradox**: a model that always says "not fraud" on the same data gets TP = 0, FN = 90, TN = 910 → accuracy 91% but recall **0**. On imbalanced data report precision, recall, F1 or AUC, not accuracy alone.

✓ Night-Before Sheet & Practice MCQs

Fact sheet:

- Supervised: labels · unsup.: clusters · RL: reward
- Neuron: $z = \Sigma wx + b$, $out = f(z)$
- Backprop (chain rule) → GD: $w \leftarrow w - \eta \partial L/\partial w$
- Sigmoid $(0,1)$ · tanh $(-1,1)$ · ReLU $[0,\infty)$
- Softmax = multiclass, sums to 1
- CNN images · RNN/LSTM series · Transformer text
- LLM = Transformer, next-token
- RAG = retrieve docs → LLM
- GAN = Generator vs Discriminator
- XAI: LIME (local), SHAP (Shapley)
- Overfit: dropout, L1/L2, more data
- $P = TP/(TP+FP)$ · $R = TP/(TP+FN)$
- $F1 = 2PR/(P+R)$

1. Best suited to time-series data?
(a) CNN (b) RNN
(c) DNN (d) ANN

2. Which is trained with backpropagation?
(a) MLP (b) CNN
(c) RNN (d) All of these

3. Output range of sigmoid:
(a) $(-1, 1)$ (b) $[0, \infty)$
(c) $(0, 1)$ (d) $(-\infty, \infty)$

4. GAN network that judges real vs fake:
(a) Discriminator (b) Generator
(c) Encoder (d) Retriever

5. TP = 80, FP = 20, FN = 10. Precision =
(a) 0.889 (b) 0.842
(c) 0.97 (d) 0.80

6. Retrieve documents, then generate:
(a) Fine-tuning (b) RAG
(c) Dropout (d) Pooling

7. Grouping customers with no labels is:
(a) supervised (b) reinforcement
(c) unsupervised (d) transfer

8. SHAP is based on:
(a) Shapley values (b) gradient descent
(c) max pooling (d) dropout

9. Output activation of a 10-class classifier:
(a) ReLU (b) Tanh
(c) Sigmoid (d) Softmax

10. Low training error, high test error:
(a) underfitting (b) good fit
(c) overfitting (d) high bias

Answers → 1-b · 2-d · 3-c · 4-a · 5-d · 6-b · 7-c · 8-a · 9-d · 10-c

Written: How can Generative AI power a government citizen-service chatbot, and how does XAI keep it transparent and accountable? 10 marks

1. **GenAI core:** an LLM reads Bangla/English queries, writes answers and summaries. 2
2. **RAG:** query → vector DB of acts and circulars → top docs + query → LLM → grounded answer with source; fewer hallucinations, no retraining. 3
3. **XAI:** cite sources; LIME/SHAP show which words drove a decision; low confidence → officer. 3
4. **Accountability & risks:** audit log, human review of high-stakes replies, bias and privacy checks. 2

Before the exam ✓: activation ranges · CNN/RNN/Transformer · P, R, F1 · RAG · GAN · agent table